

C#.NET



Multithreading

Part-3

Mr. Bangar Raju



Understanding Thread Class Constructors in C#.NET

By Albert Harmon Published April 12, 2017 ⌚ 3 min read [Technology](#)



TLDR This blog post explores the constructors of the Thread class in C# .NET, detailing how to create and manage threads using delegates, including examples of parameterized and non-parameterized methods.

Welcome to this comprehensive guide on the constructors of the Thread class in C#. This is part three of our multithreading series, where we will delve into how to create and manage threads effectively.

Multithreading is a powerful feature in C# that allows concurrent execution of code. In this post, we will focus on the constructors available in the Thread class and how to utilize them to run methods in separate threads.

Creating a Simple Method

To demonstrate the use of threads, we will first create a simple static method:

```
static void Test() {  
    for (int i = 1; i <= 100; i++) {  
        Console.WriteLine(i);  
    }  
}
```

This method prints numbers from 1 to 100. Normally, we would call this method directly from the main thread. However, our goal is to execute it using a child thread.

Instantiating the Thread Class

To call our method using a thread, we need to create an instance of the Thread class. First, we must import the necessary namespace:

```
using System.Threading;
```

Next, we create an instance of the Thread class:

```
Thread t1 = new Thread(Test);
```

Understanding Thread Constructors

The Thread class has four constructors, but we will focus on the first two:

1. `Thread(ThreadStart start)` - This constructor takes a delegate of type `ThreadStart`.
2. `Thread(ThreadStart start, bool isBackground)` - This constructor also takes a `ThreadStart` delegate and a boolean indicating if the thread is a background thread.

What is a Delegate?

A delegate in C# is a type-safe function pointer that allows methods to be passed as parameters. The `ThreadStart` delegate is a specific type of delegate that does not take any parameters and does not return a

value. The signature of the method we want to call must match the signature of the delegate.

Binding a Method to a Delegate

To bind our method to the `ThreadStart` delegate, we can instantiate the delegate as follows:

```
ThreadStart obj = new ThreadStart(Test);
```

Now, we can pass this delegate to the `Thread` constructor:

```
t1 = new Thread(obj);
```

Implicit vs. Explicit Delegate Creation

When we pass the method name directly to the `Thread` constructor, the Common Language Runtime (CLR) implicitly creates the delegate for us. However, we can also explicitly create the delegate as shown above. Both approaches yield the same result, but understanding the difference is crucial for deeper knowledge of C# threading.

Parameterized Thread Start

If we want to pass parameters to our method, we need to use the `ParameterizedThreadStart` delegate. This delegate takes an object as a parameter. Here's how we can modify our method to accept a parameter:

```
static void Test(object max) {  
    int limit = Convert.ToInt32(max);  
    for (int i = 1; i <= limit; i++) {  
        Console.WriteLine(i);  
    }  
}
```

Creating a Parameterized Thread

To create a parameterized thread, we instantiate the `ParameterizedThreadStart` delegate:

```
ParameterizedThreadStart obj = new ParameterizedThreadStart(  
    Thread t2 = new Thread(obj);
```



When starting the thread, we can pass the parameter:

```
t2.Start(50);
```

This will print numbers from 1 to 50. However, since the parameter is of type object, we must ensure that we pass an integer value to avoid runtime errors.

Conclusion

In this post, we explored the constructors of the Thread class in C#. We learned how to create threads using both non-parameterized and parameterized methods, and we discussed the importance of delegates in this process. Understanding these concepts is essential for effective multithreading in C# applications.

Thank you for reading! For more tutorials and insights, stay tuned for our next video.